

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Franco Guerrero Gómez

9 de septiembre de 2026

22:09

1. Introducción

En este reporte se presenta el estudio experimental del rendimiento práctico de algoritmos de ordenamiento de arreglos unidimensionales (Merge Sort $\Theta(n \log n)$, Quick Sort $O(n^2)$, Patience Sort $O(n^2)$ y sort $\Theta(n \log n)$) y de algoritmos de multiplicación de matrices cuadradas (Naive $\Theta(n^3)$ y Strassen $\Theta(n^{2.81})$).

Esto se realizó mediante la implementación de cada algoritmo en C++ y la ejecución de pruebas controladas para medir su rendimiento en términos de tiempo de ejecución y consumo máximo de memoria RAM (*Peak RSS*), el objetivo principal de tomar estas mediciones es contrastar las cotas asintóticas teóricas con las complejidades observadas en la ejecución de distintos casos, considerando factores como la arquitectura de memoria, la profundidad de recursión y las condiciones específicas de los archivos de entrada.

2. Entorno experimental

Los experimentos fueron ejecutados en un entorno Linux sobre WSL (Windows Subsystem for Linux), utilizando un procesador AMD Ryzen 3 7320U with Radeon Graphics (2.40 GHz), Sistema operativo de 64 bits, procesador x64 y 8 GB de memoria RAM. Los programas principales fueron implementados en C++17 y compilados con g++ utilizando el nivel de optimización predeterminado vía `Makefile`.

La medición de tiempo se realizó con la librería estándar `<chrono>` (`high_resolution_clock`), mientras que el consumo de memoria *Peak RSS* se registró mediante la función `getrusage()` de la cabecera

POSIX `<sys/resource.h>`.

Para las pruebas de ordenamiento se generaron arreglos unidimensionales mediante un script de python con tamaños $n \in \{10^1, 10^3, 10^5, 10^7\}$, considerando tres tipos de orden inicial (*ascendente*, *descendente*, *aleatorio*) y dos dominios de valores ($D1 : [0, 9]$ y $D7 : [0, 10^7]$), con lo cual se construye el dataset de arreglos.

Para las pruebas de multiplicación de matrices, se generaron matrices mediante un script de python utilizando dimensiones $n \in \{2^4, 2^6, 2^8, 2^{10}\}$ en tres estructuras (*densa*, *diagonal*, *dispersa*) sobre los dominios $D0 : \{0, 1\}$ y $D10 : [0, 9]$, con lo cual se construye el dataset de matrices.

Las visualizaciones graficas fueron generadas mediante scripts de Python usando las libreria pandas y seaborn.

3. Resultados y Análisis

3.1. Multiplicación de Matrices Cuadradas

3.1.1. Tiempo de Ejecución

En el análisis de los resultados experimentales, se observa que el algoritmo de Strassen supera al algoritmo de Naive en tiempo de ejecución para matrices densas, en la escala logarítmica (10^{-2} a 10^3 ms), Strassen muestra tiempos superiores a Naive en la gran mayoría de las dimensiones evaluadas (de $n = 64$ a $n = 1024$). Solo en $n = 16$ se aprecia un tiempo ligeramente menor.

Ademas, se observa que el tiempo de ejecución de Strassen crece a un ritmo más lento que el de Naive, lo que indica que la complejidad asintótica de Strassen es menor, como se aprecia en la Figura 1:

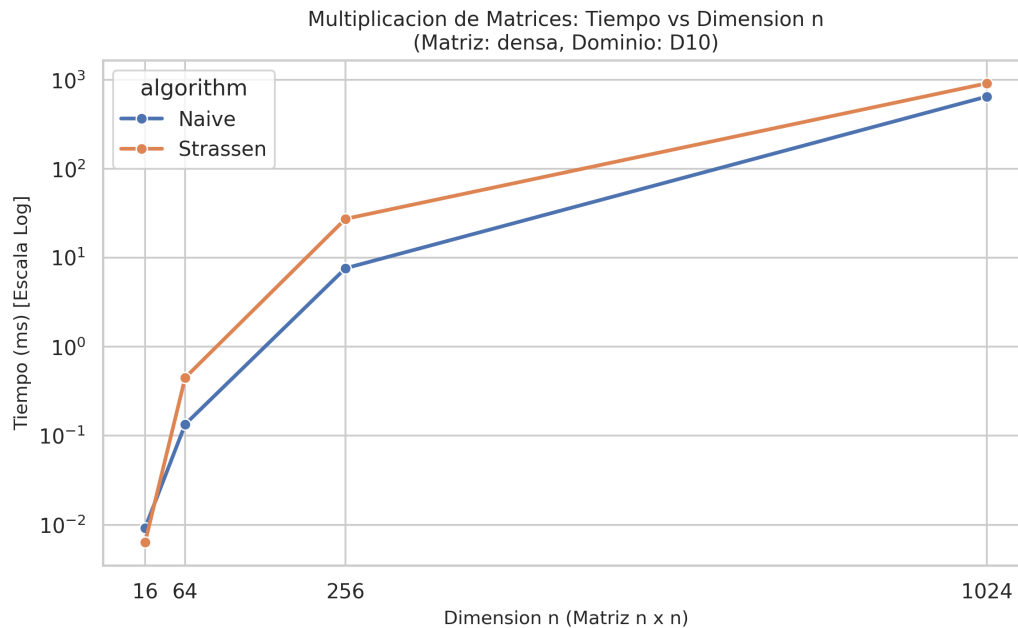


Figura 1: Tiempo de ejecución vs. dimensión n para matrices densas en dominio D10.

A pesar de que teóricamente Strassen tiene una menor complejidad asintótica $O(n^{2.81})$ frente a la multiplicación realizada por Naive con complejidad asintótica $O(n^3)$, Strassen tiene un mayor tiempo de ejecución que Naive, esto se debe a que este primer algoritmo requiere guardar submatrices intermedias que participan en un proceso de suma y resta de matrices que busca reducir el número de llamadas recursivas, lo que termina provocando un overhead de tiempo de ejecución para matrices que no son lo suficientemente grandes.

3.1.2. Consumo de Memoria

Respecto al consumo de memoria (Peak RSS) utilizado por los algoritmos, se observa que el algoritmo de Strassen escala de forma drástica al aumentar la dimensión, alcanzando aproximadamente 6800 KB para $n = 1024$, mientras que el método Naive se mantiene cercano a 1300 KB, creciendo de forma lineal, como se muestra en la Figura 2:

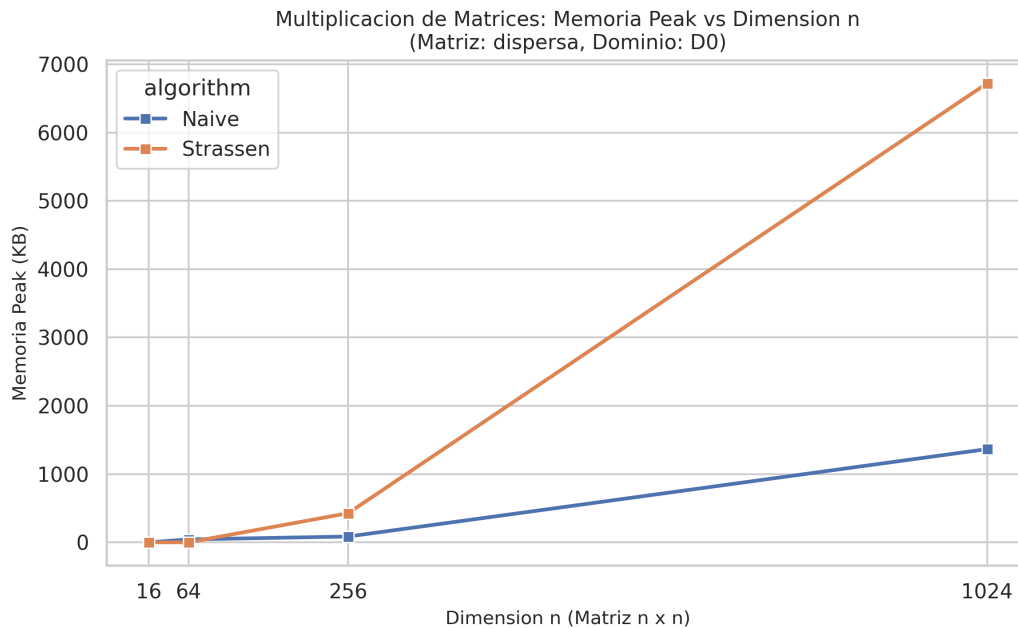


Figura 2: Uso de memoria Peak RSS vs. dimensión n para matrices dispersas en dominio D0.

Este comportamiento se debe a la creación de 10 submatrices intermedias ($M_1 \dots M_7, A_{11}$, etc.) por cada nivel de la pila recursiva en el algoritmo de Strassen, resultando en un impacto de memoria con un factor constante elevado en comparación a la estructura estática del método iterativo que utiliza Naive.

3.2. Ordenamiento de Arreglos Unidimensionales

3.2.1. Tiempo de Ejecución

En el análisis de los resultados experimentales, se observa que los algoritmos de ordenamiento Quick Sort y Merge Sort presentan un tiempo de ejecución superior al de los algoritmos Patience Sort y Sort para arreglos ordenados de manera descendente, especialmente el algoritmo Quick Sort, del cual no se tomaron datos para arreglos de tamaño $n = 10^7$, debido al desbordamiento de pila (*Stack Overflow*) y un error de segmentación (*Segmentation Fault*) al superar el límite del *call stack* del sistema operativo (8MB)..

El algoritmos Sort fue el de mejor desempeño en cuanto a tiempo de ejecución, mientras que Patience Sort y Merge Sort mostraron un desempeño comparable a la par de estabilidad en todas las configuraciones, manteniendo una curva asintótica parecida a $O(n \log n)$, como se aprecia en la Figura 3:

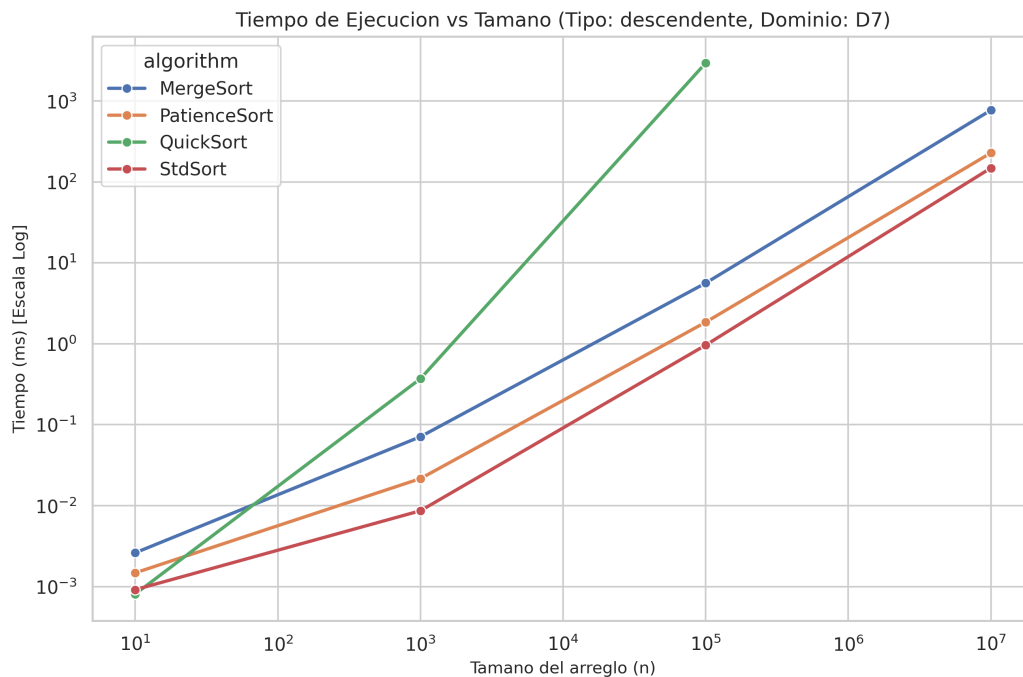


Figura 3: Tiempo de ejecución vs. tamaño n para arreglo descendente en dominio D7.

El comportamiento observado en los algoritmos de ordenamiento coincide con la complejidad asintótica teórica de cada uno, donde Quick Sort, con $O(n^2)$ cae en el peor caso por tratarse de un arreglo ordenado, causando una recursion completamente desbalanceada, mientras que Patience Sort, que también cuenta con tiempo $O(n^2)$ presenta un tiempo promedio bastante estable parecido a $O(n \log n)$, el cual incluso supera en la practica al algoritmo Merge Sort, con tiempo estable $\Theta(n \log n)$, ambos tienen una curva muy similar al algoritmo Sort, que también tiene un tiempo promedio de $\Theta(n \log n)$ y un comportamiento estable en todos los rangos de n evaluados.

Este último algoritmo logra su eficiencia gracias a su naturaleza híbrida IntroSorty optimizaciones a nivel de compilador, logrando los menores tiempos de ejecución en comparación a los otros algoritmos con complejidad asintotica similar.

3.2.2. Consumo de Memoria

Respecto al consumo de memoria (Peak RSS) utilizado por los algoritmos, se observa que el algoritmo de Merge Sort escala de forma drástica al aumentar el tamaño del arreglo, alcanzando aproximadamente 12000 KB para $n = 10^7$, mientras que el método Sort se mantiene cercano a 1800 KB, creciendo de forma lineal.

Por otro lado, los algoritmos Sort y Quick Sort muestran un comportamiento imperceptible de uso de memoria, permaneciendo sobre la línea de 0 KB, como se muestra en la Figura 4:

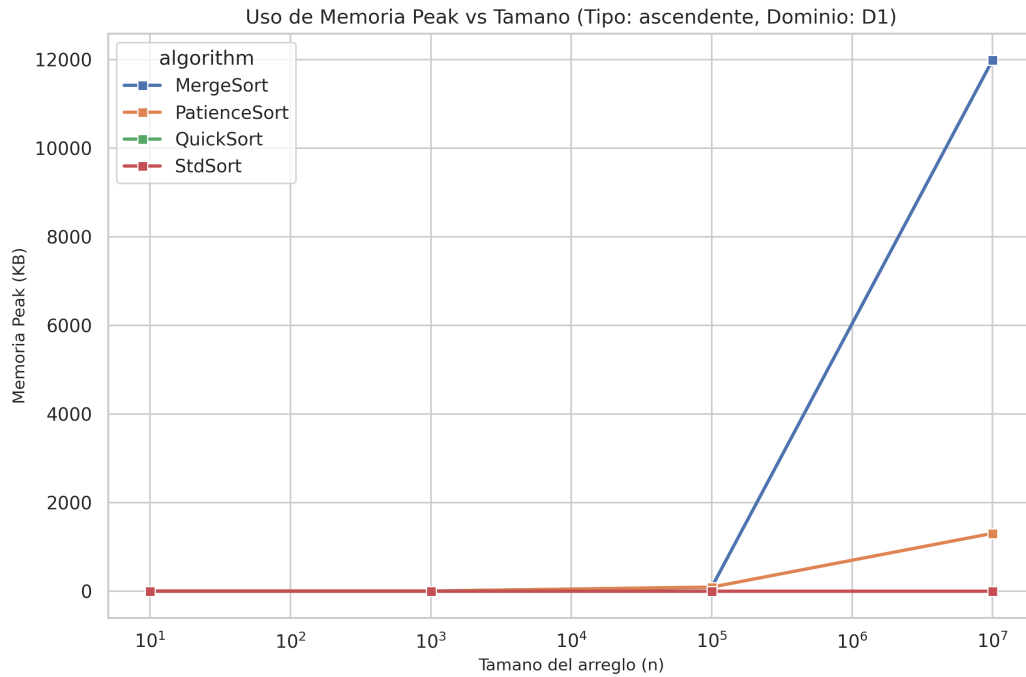


Figura 4: Uso de memoria Peak RSS vs. tamaño n para arreglo ascendente en dominio D1.

El comportamiento observado en los algoritmos de ordenamiento coincide con la complejidad asintótica teórica de cada uno, donde Merge Sort exhibe una asignación auxiliar proporcional a $O(n)$ debido a los vectores temporales en la fase de fusión, mientras que Sort mantiene un uso de memoria mínimo, con $O(1)$ en el consumo de memoria, realizando el ordenamiento in-place sin requerir espacio adicional significativo.

En el caso de Quick Sort y Sort, ambos son algoritmos in-place. La única memoria adicional consumida proviene del marco de llamadas de la pila de recursión, lo cual es prácticamente imperceptible a nivel del sistema operativo en términos de Peak RSS.

4. Conclusiones

A partir de los resultados obtenidos, se concluye que en los algoritmos de multiplicación de matrices, Naive resulta más eficiente que Strassen en tiempo de ejecución para todas las dimensiones evaluadas hasta $n = 1024$, evidenciando de que la sobrecarga de submatrices intermedias de Strassen ralentiza su desempeño y genera además un consumo de memoria notablemente mayor. En el ordenamiento de arreglos, el algoritmo Sort se consolida como la alternativa más robusta y eficiente, tanto en tiempo de ejecución como en uso de memoria auxiliar. También se demostró la alta sensibilidad que tiene el algoritmo QuickSort con partición de Lomuto ante entradas ordenadas, donde su tiempo de ejecución empeora drásticamente de $O(n \log n)$ a $O(n^2)$, mientras que MergeSort garantiza estabilidad temporal ante cualquier escenario de datos a costa de un costo espacial estricto $O(n)$ frente a las soluciones in-place.